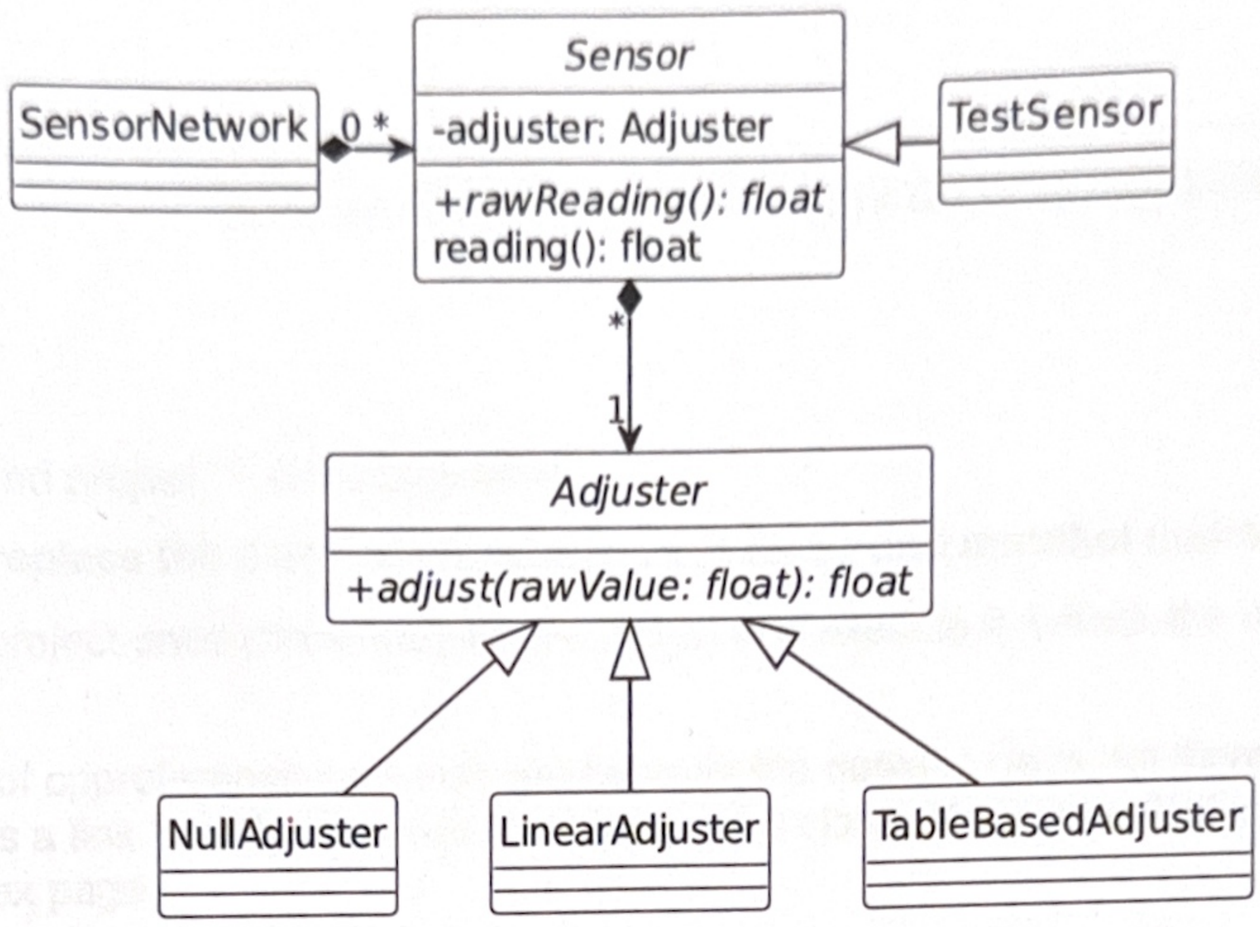
h_da	Fachbereich Elektrotechnik und Informationstechnik	Advanced Programming Techniques (APT) Module Exam SoSe2025 (PO2019) Prof. Dr. Lipp

Topic of this exam is the (partial) modelling of a sensor network (see picture below).



The diagram shows the classes, their relationships and only a subset of attributes and methods related to sensor calibration. You can find all attributes and methods in the class definitions. The comments there also describe the classes and methods in detail.

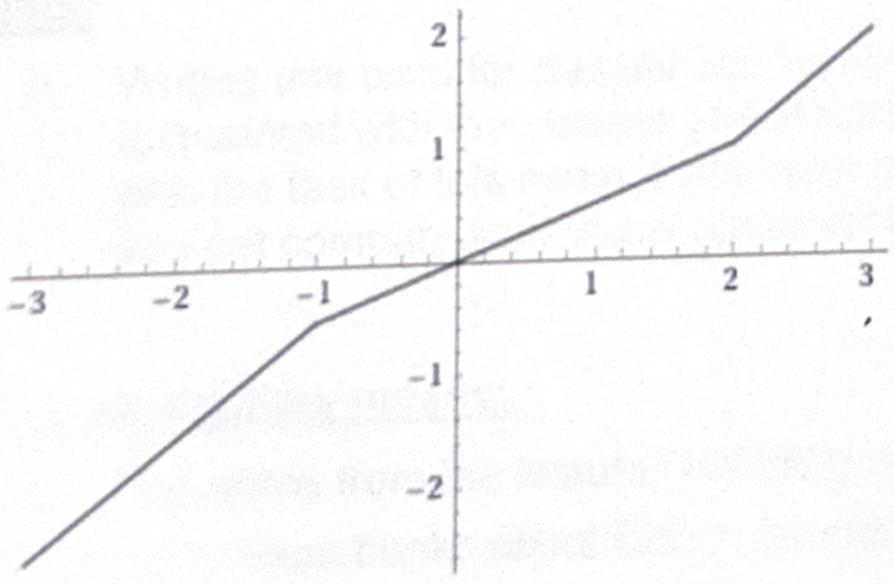
Exercise

Implement the classes and their methods as described in the header files.

Adding private methods in order to avoid redundant code is allowed. Adding to or modifying the classes' attributes is forbidden. Adding to or modifying the public or protected methods is forbidden as well.

Implement the tests outlined in file "test.cpp". Note that "assert that …" means that you have to write one or more `assertTrue(...)`-statements. Unless there is an error, running your tests must not produce any output on the console.

The Illustration mentioned in TableBasedAdjuster::adjust:



Note

The points that you get for implementing a particular method can be found in the methods comment. The total number of points that you can reach is 84.

h_da	Fachbereich Elektrotechnik und Informationstechnik	Advanced Programming Techniques (APT) Module Exam WiSe2024 (PO2019) Prof. Dr. Lipp

Name	Matr.-Nr.	Computer	Signature (*)

(*) I hereby declare that I am able and willing to take the examination with my signature.

Rules:

- Use the prepared project "Exam-2024WiSe".
- **In `main.cpp` replace the place holders with your name and matrikel number.**
- Compile your project once (there may be warnings) and execute it. Check the output.
- Access to
- The snapshot of cppreference.com that you have on the desktop does not have a search facility. However, it has a link "std Symbol Index" which comes close. Make sure that you have found the link on the index page.
- Create all classes in the folder „`myCode`".
- "`using namespace`" statements in header files are forbidden.
- Code that implements methods (and functions) must be written in the proper *.cpp-file (no implementation code in *.h-files).
- Not using defined methods when appropriate (copying code instead) results in a reduction of points.
- Leaving automatically generated, not required code such as default constructors in the code results in a reduction of points.
- Code will only be graded up to the first compiler error.
- Code in comments will not be graded.
- Answers to questions will only be accepted when provided in the specified comments. Answers found elsewhere in your code will gain no points, even if they are correct.

Hints:

- Writing unit tests for classes can be done before the implementation and is a possibility to get acquainted with the classes and the problem domain. The effort required to familiarize yourself with the task of this exam is therefore taken into account in the task of writing unit tests where you get comparatively many points for little code.

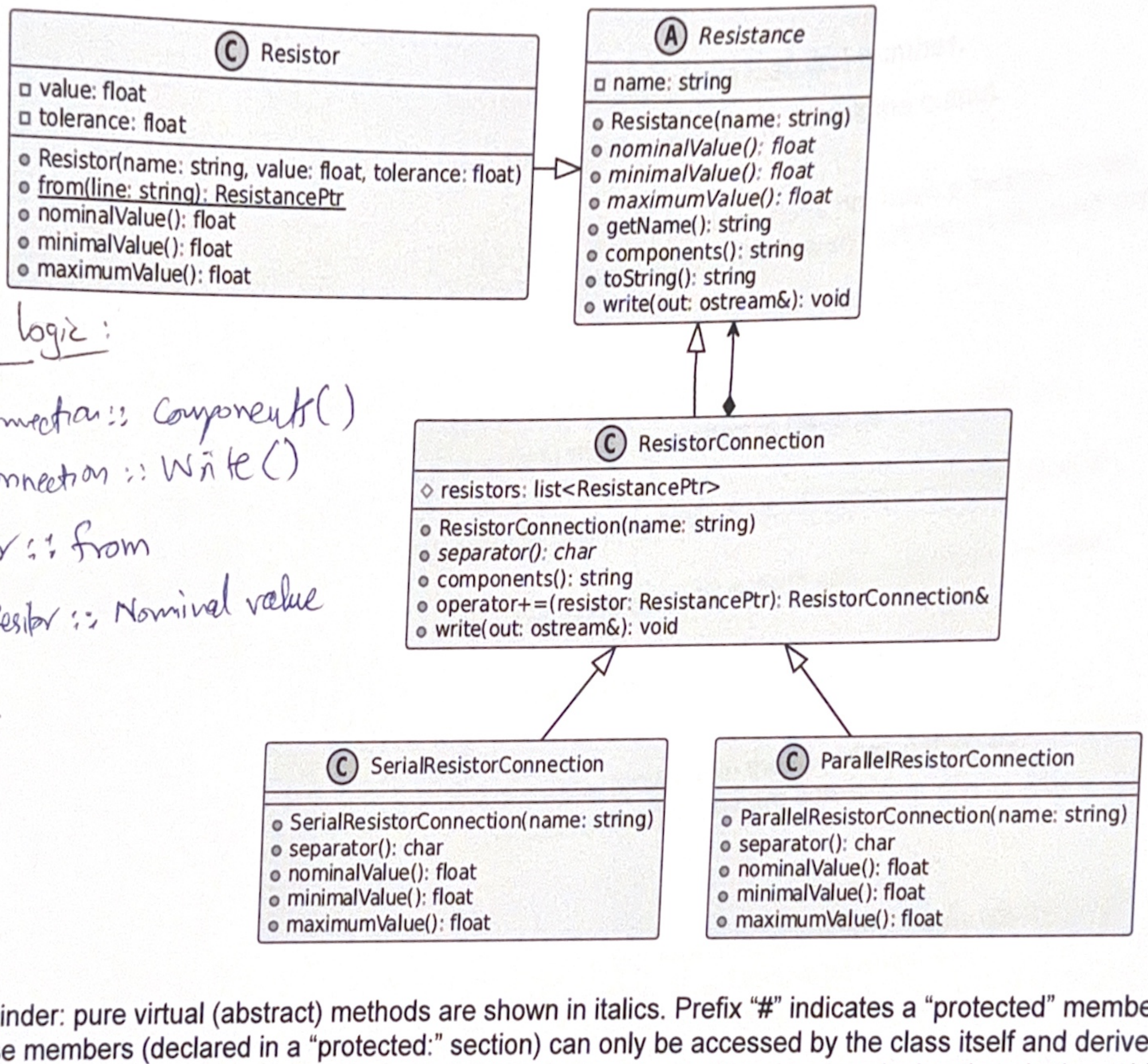
Allowed auxiliary means:

- The slides from the lecture (including print-outs of the uploaded code samples)
- Up to three books about C/C++ programming
- A dictionary.

Communication with other participants or using unauthorized auxiliary means, including but not limited to electronic devices such as smart phones, smart watches calculators, will result in immediate termination of the exam and a 0 point grading.

h_da	Fachbereich Elektrotechnik und Informationstechnik	Advanced Programming Techniques (APT) Module Exam WiSe2024 (PO2019) Prof. Dr. Lipp

Topic of this exam is the modelling of resistor networks (see picture below). The general interface to any resistor network is specified by class `Resistance`. An instance may simply be a physical resistor (class `Resistor`). Another possibility is to have a connection of two or more resistors, either a serial connection or a parallel connection.



Important logic:

1.) ResistorConnection :: components()
ResistorConnection :: write()

2.) Resistor :: from
Parallel

3.) ~~Serial~~ Resitor :: Nominal value

4.) tests.

Reminder: pure virtual (abstract) methods are shown in italics. Prefix "#" indicates a "protected" member. These members (declared in a "protected:" section) can only be accessed by the class itself and derived classes. (If you have problems implementing this, you can also put the member declarations in the public section.)

Exercise

Implement the missing methods according to their description in the header files. Adding to or modifying the classes' attributes or methods is forbidden, except where stated in the comments.

Implement the tests outlined in file "tests.cpp". Note that "assert that …" means that you have to write one or more `assertTrue(...)`-statements. Unless there is an error, running your tests must not produce any output on the console.

Note

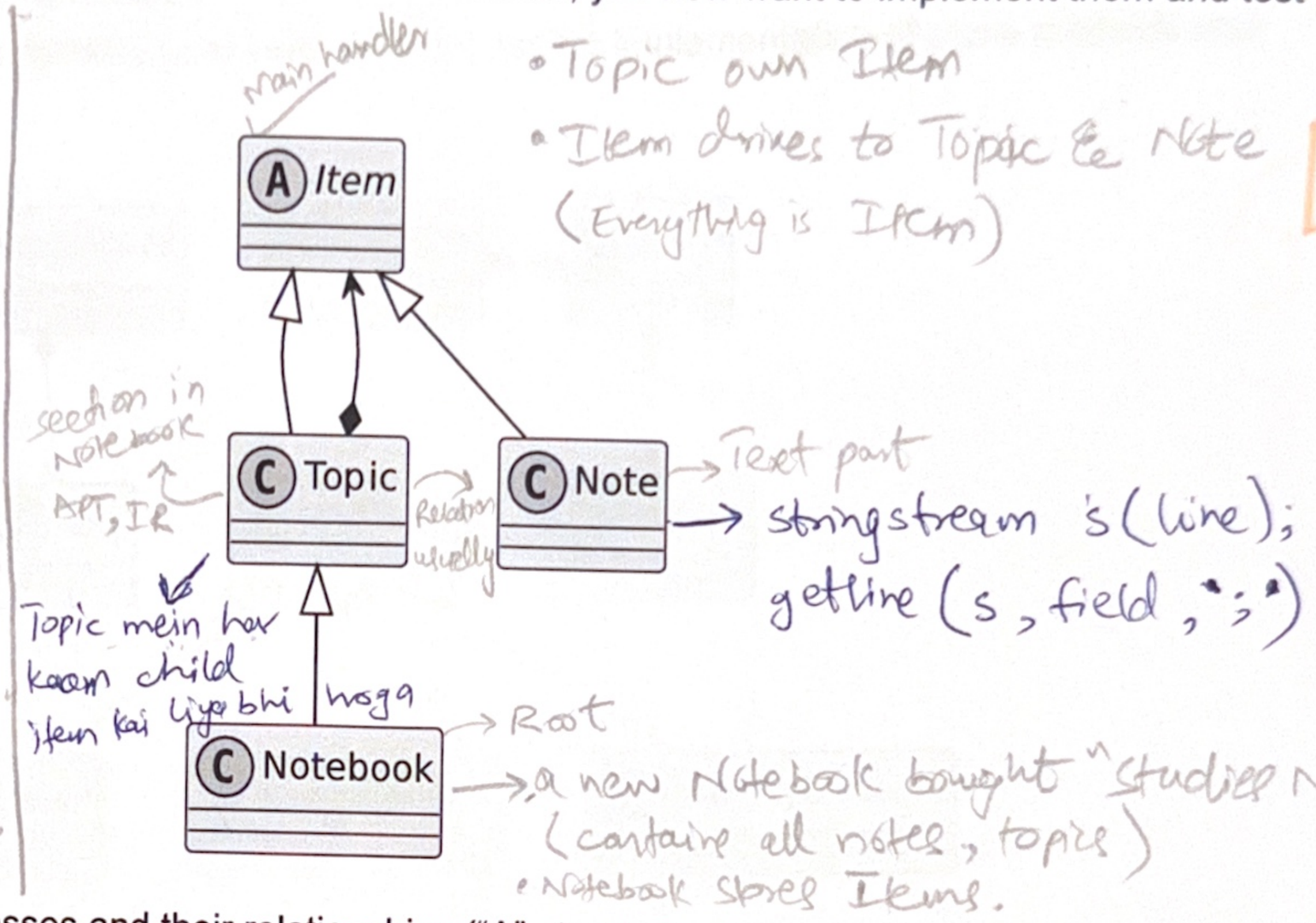
The points that you get for implementing a particular method or test can be found in the methods' comments. The total number of points that you can achieve is 100.

* Notebook is a Topic
* Notes always belong to a Topic

h_da	Fachbereich Elektrotechnik und Informationstechnik	Advanced Programming Techniques (APT) Module Exam SoSe2024 (PO2019) Prof. Dr. Lipp

You want to implement a simple notebook application. The notebook consists of `Topics` and `Notes`, which are each a kind of `Item`. `Notes` have content (text). `Topics` contain `Notes` or other (sub-)`Topics`, i.e. `Topics` form an instance hierarchy. Thinking about this, you have found that the `Notebook` itself is actually a special `Topic` with some methods that make sense for this "top-level `Topic`" only. Having defined the classes and some methods, you now want to implement them and test the basic functions.

"Studies Notebook"
- Note: "Work carefully"
  - "Plan Ahead"
- Topic: "APT"
  - Note: Learn C++
    - "Remember..."
  - Note: "Learn Libraries"
- Topic: "Other"
  - Note: "Learn Something"
  - Note: "Learn more"
- Note: "Finish the Studies"



- Topic own Item
- Item drives to Topic & Note (Everything is Item)

4

The diagram shows the classes and their relationships ("A" stands for abstract class). You can find the attributes and methods in the class definitions. The comments provided there also describe the classes and methods in detail.

Exercise

Implement the missing methods according to their description in the header files.

Adding to or modifying the classes' attributes or methods is forbidden.

Implement the tests outlined in file "tests.cpp". Note that "assert that ..." means that you have to write one or more `assertTrue(...)`-statements. Unless there is an error, running your tests must not produce any output on the console.

Note

The points that you get for implementing a particular method or test can be found in the methods' comments. The total number of points that you can achieve is 100.

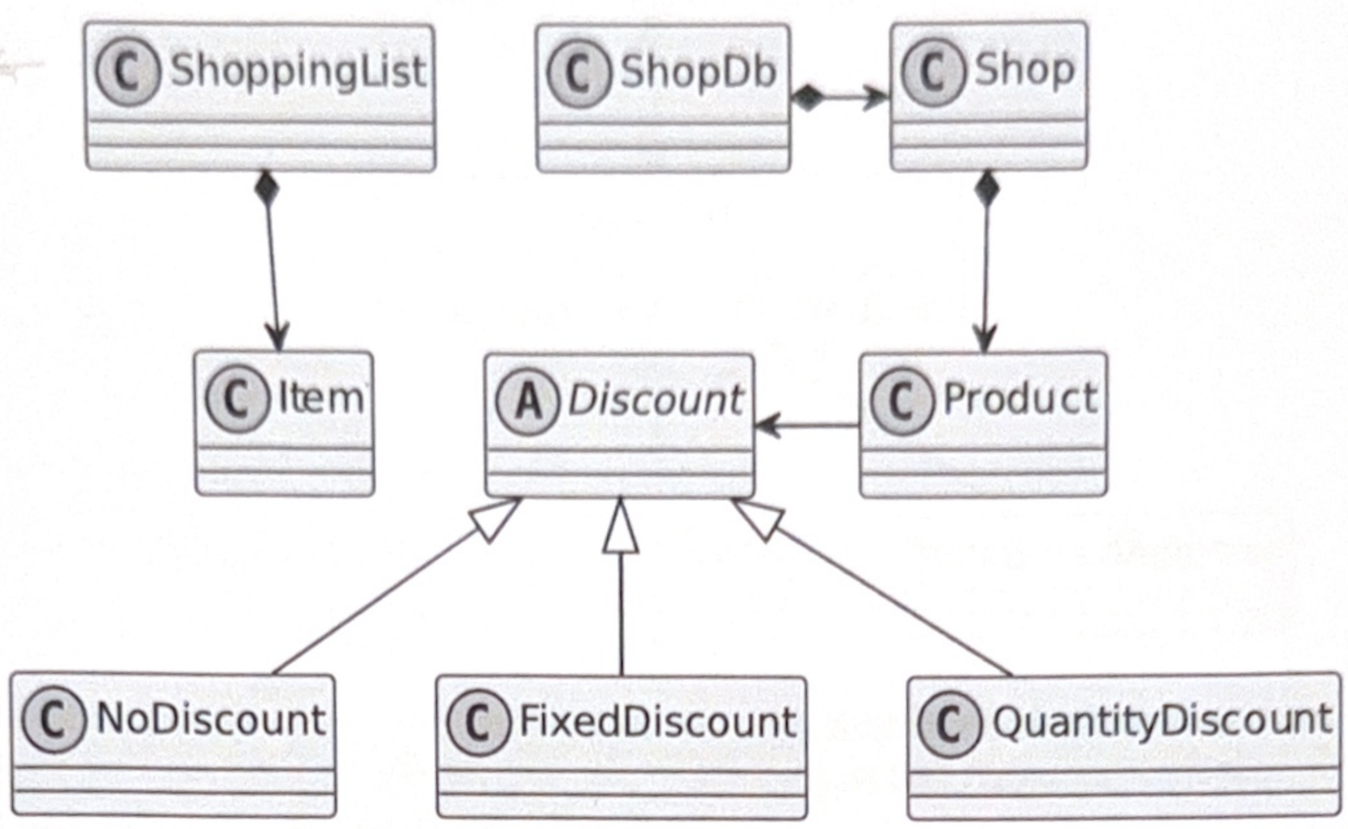
Important Logic:
1.) structer of printItem's
2.) Structre of writeCSV's
3.) structre of fromCSV's
4.) In Topic → printItem → writeCSV: child Items to handle as well.
5.) Topic:: fromCSV
6.) Notebook:: loadfromCSV
7.) Tests → streams to handle (stringstream (scoping), ostringstream, ofstream, ifstream)

ostringstream out.
writeCsv(out, parent)
out.str()
↳ gives content from out pipeline.

h_da	Fachbereich Elektrotechnik und Informationstechnik	Advanced Programming Techniques (APT) Module Exam WiSe2023 (PO2019) Prof. Dr. Lipp

Considering inflation, you have the idea to build an app that compares prices at different shops. It takes a shopping list as input and calculates the price for the purchase, using data from a "shop database". The shop database manages information about shops, the products that they offer and discounts that the shops may grant for certain products.

The classes for a simple demonstrator have been defined, but the implementation of some methods and tests is still missing.



The diagram shows the classes and their relationships. You can find the attributes and methods in the class definitions. The comments there also describe the classes and methods in detail.

Exercise

Implement the stub methods ("TODO") according to their description in the header files.

Adding to or modifying the classes' attributes or methods is forbidden.

Implement the tests outlined in file "tests.cpp". Note that "assert that …" means that you have to write one or more `assertTrue(...)`-statements. Unless there is an error, running your tests must not produce any output on the console.

Note

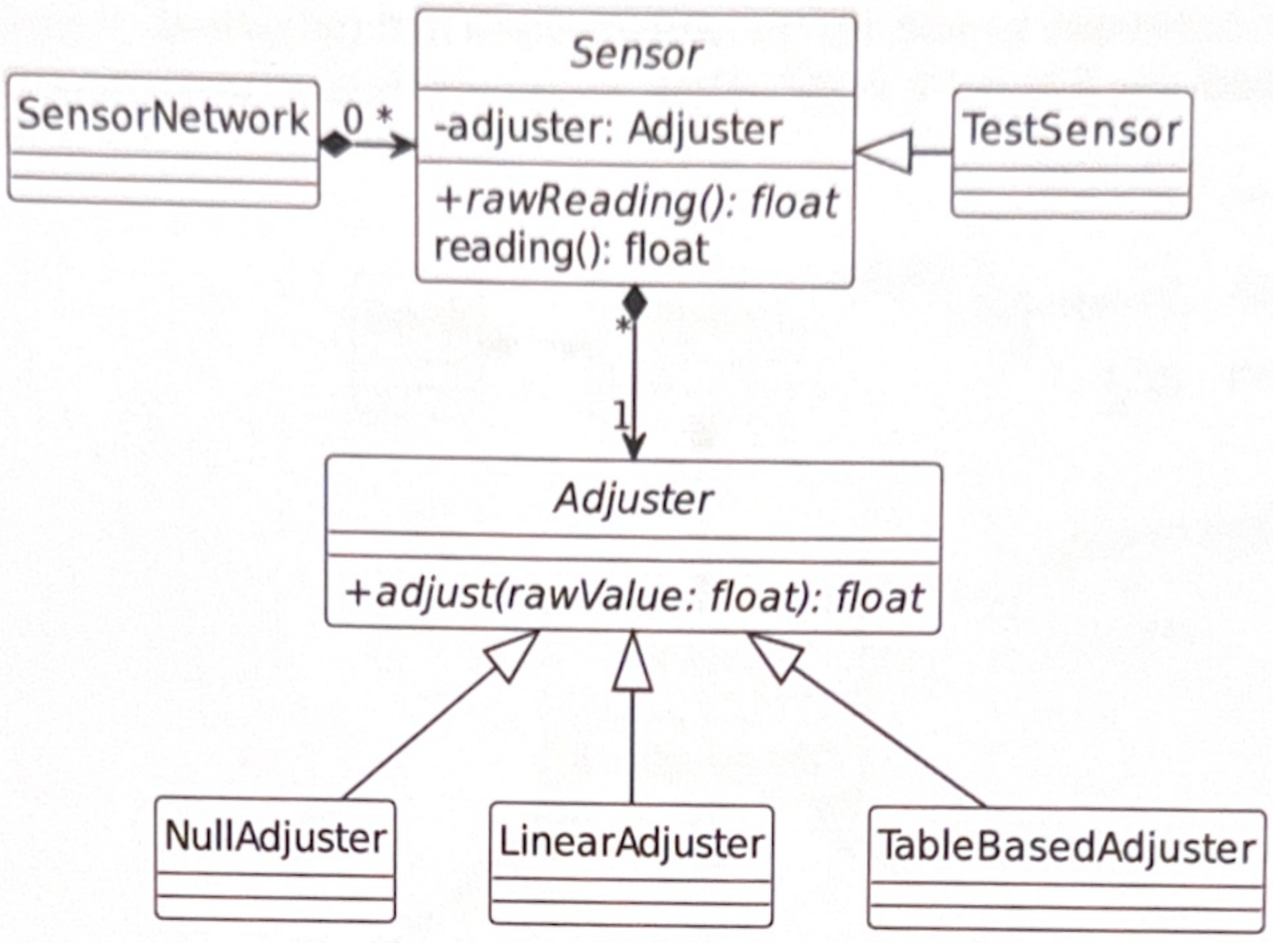
The points that you get for implementing a particular method or test can be found in the methods comment. The total number of points that you can reach is 100.

Important Logic:

1) Product :: priceFor
2) Shop :: CalculatePurchase
3) QuantityDiscount :: discountFor → use 0.0f for comparasion.

h_da	Fachbereich Elektrotechnik und Informationstechnik	Advanced Programming Techniques (APT) Module Exam SoSe2023 (PO2019) Prof. Dr. Lipp

Topic of this exam is the (partial) modelling of a sensor network (see picture below).



The diagram shows the classes, their relationships and only a subset of attributes and methods related to sensor calibration. You can find all attributes and methods in the class definitions. The comments there also describe the classes and methods in detail.

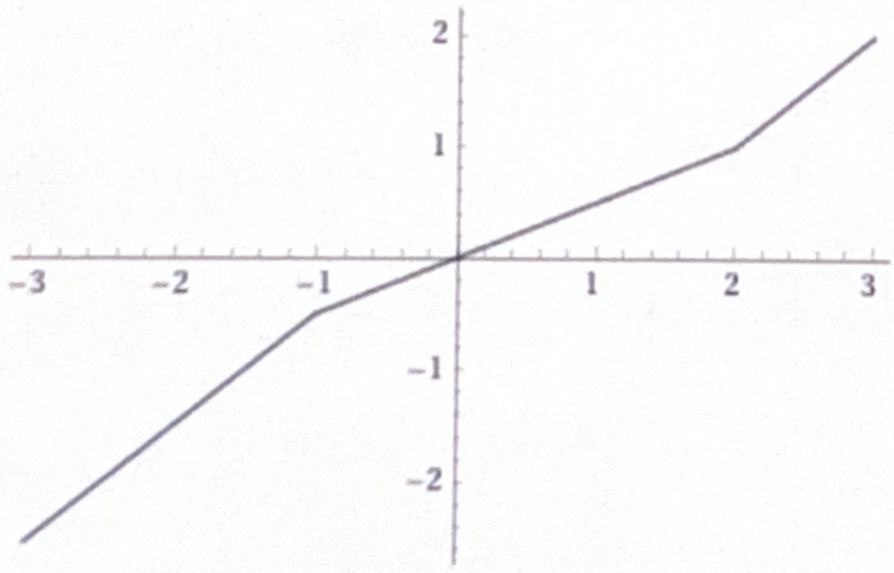
Exercise

Implement the classes and their methods as described in the header files.

Adding private methods in order to avoid redundant code is allowed. Adding to or modifying the classes' attributes is forbidden. Adding to or modifying the public or protected methods is forbidden as well.

Implement the tests outlined in file "test.cpp". Note that "assert that …" means that you have to write one or more `assertTrue(...)`-statements. Unless there is an error, running your tests must not produce any output on the console.

The Illustration mentioned in TableBasedAdjuster::adjust:

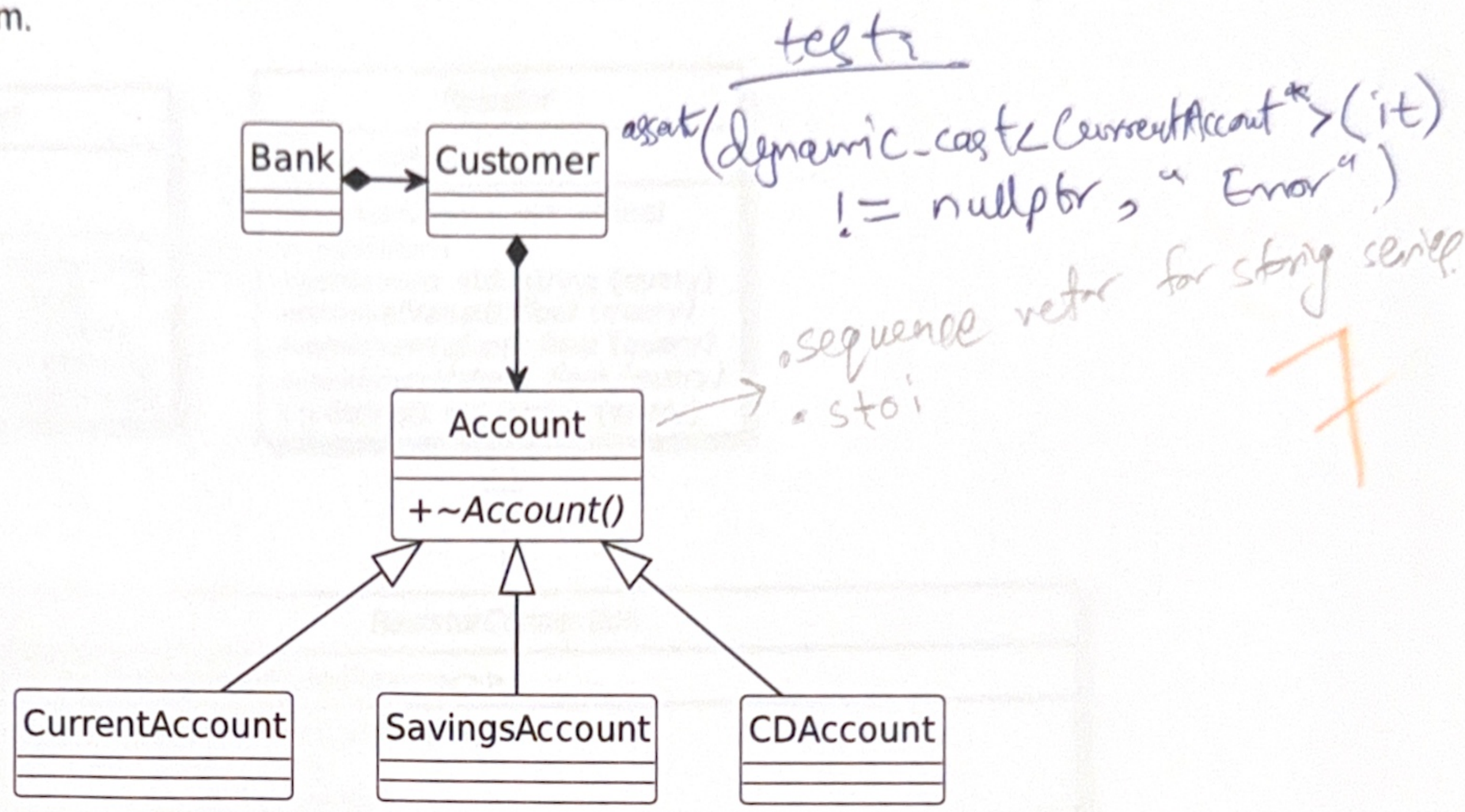


Note

The points that you get for implementing a particular method can be found in the methods comment. The total number of points that you can reach is 84.

h_da	Fachbereich Elektrotechnik und Informationstechnik	Advanced Programming Techniques (APT) Module Exam WiSe2022 (PO2019) Prof. Dr. Lipp

Topic of this exam is the modelling of some data related to customers of a bank (see picture below). A bank has customers who have one or more accounts. Accounts can be of different type. The current account (a.k.a. checking account, German: *Girokonto*), usually only used for transferring money. A savings account (German: *Sparkonto*) that earns interest on the money deposited in it. A certificate of deposit (CD) account (German: *Festgeldkonto*) that earns higher interest if you leave your money in the account for a fixed term.



The diagram shows only the classes and their relationships. You'll find the attributes and methods in the class definitions.

Exercise

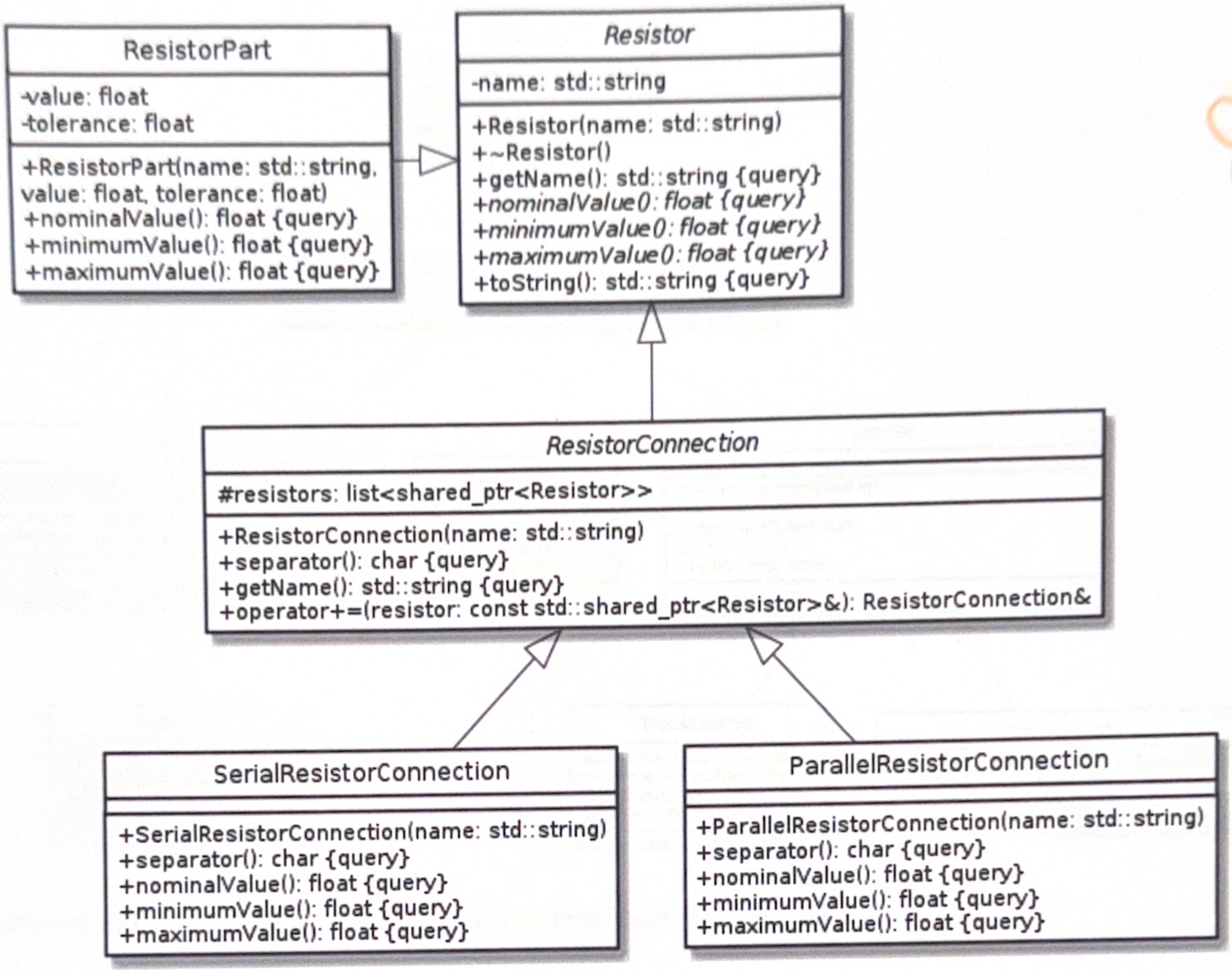
Implement the classes and their methods as described in the header files.

Adding private methods in order to avoid redundant code is allowed. Adding to or modifying the classes' attributes is forbidden. Adding to or modifying the public or protected methods is forbidden as well.

Implement the tests outlined in file "test.cpp". Note that "assert that …" means that you have to write one or more `assertTrue(...)`-statements. Unless there is an error, running your tests must not produce any output on the console.

h_da	Fachbereich Elektrotechnik und Informationstechnik	Advanced Programming Techniques (APT) Module Exam SoSe2022 (PO2019) Prof. Dr. Lipp

Topic of this exam is the modelling of resistor networks (see picture below). The general interface to any resistor network is specified by class Resistor. An instance may simply be a physical resistor (class `ResistorPart`). Another possibility is to have a connection of two or more resistors, either a serial connection or a parallel connection.



Reminder: pure virtual (abstract) methods are shown in italics.

Prefix "#" indicates a "protected" member. These members (declared in a "protected:" section) can only be accessed by the class itself and derived classes. (If you have problems implementing this, you can also put the member declarations in the public section.)

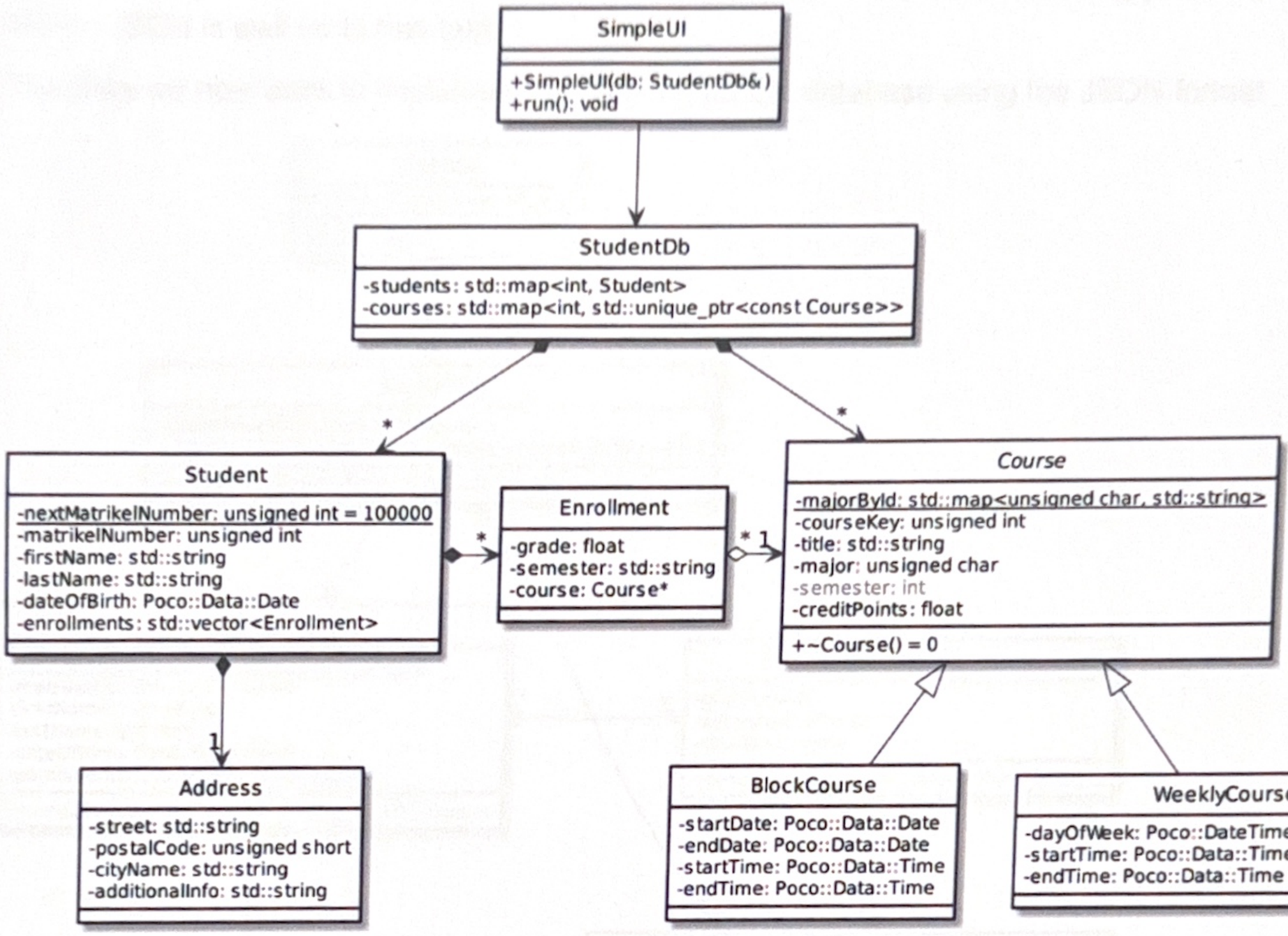
(The line break in `ResistorPart::ResistorPart(…)` has no significance, it has been inserted for a more compact layout.)

Exercise 1 (51 points)

a) Define class `Resistor` and implement its methods. The destructor does nothing, but defining it avoids a compiler warning and is best practice for abstract classes. The method `toString` returns the result from invoking `getName()` and `nominalValue()` formatted as: "<name>=<value> Ohm", e. g. "`R1=42 Ohm`" (no line break) (see sample output below). To save you a lot of typing, add a line "`typedef std::shared_ptr<Resistor> ResistorPtr;`" in the header file. It allows you to use `ResistorPtr` wherever you'd have to write `std::shared_ptr<Resistor>` without the typedef.	9

1 Add attribute semester to Class Course

As already mentioned as an aside during the lecture, a course takes place in a particular semester. It therefore makes sense to add this information as an attribute `semester` in class `Course`.



Use your submitted solution for lab 3 as start project for this task.

1.1 Extended constructor and getter (38)

While the attribute has the characteristics of a string (“WiSe2021”, “SoSe2022”, “WiSe2022”, …) an integer value is used for storage efficiency and the mapping takes place in the constructor and the getter method.

Add the private attribute in your class definition and extend the constructor and implement the getter method as specified below. Because the attribute was introduced in WiSe2021, this particular semester is mapped to 0. However, in order to be able to also handle historical data, the methods must handle negative values correctly (e. g. “SoSe2021” must map to -1).

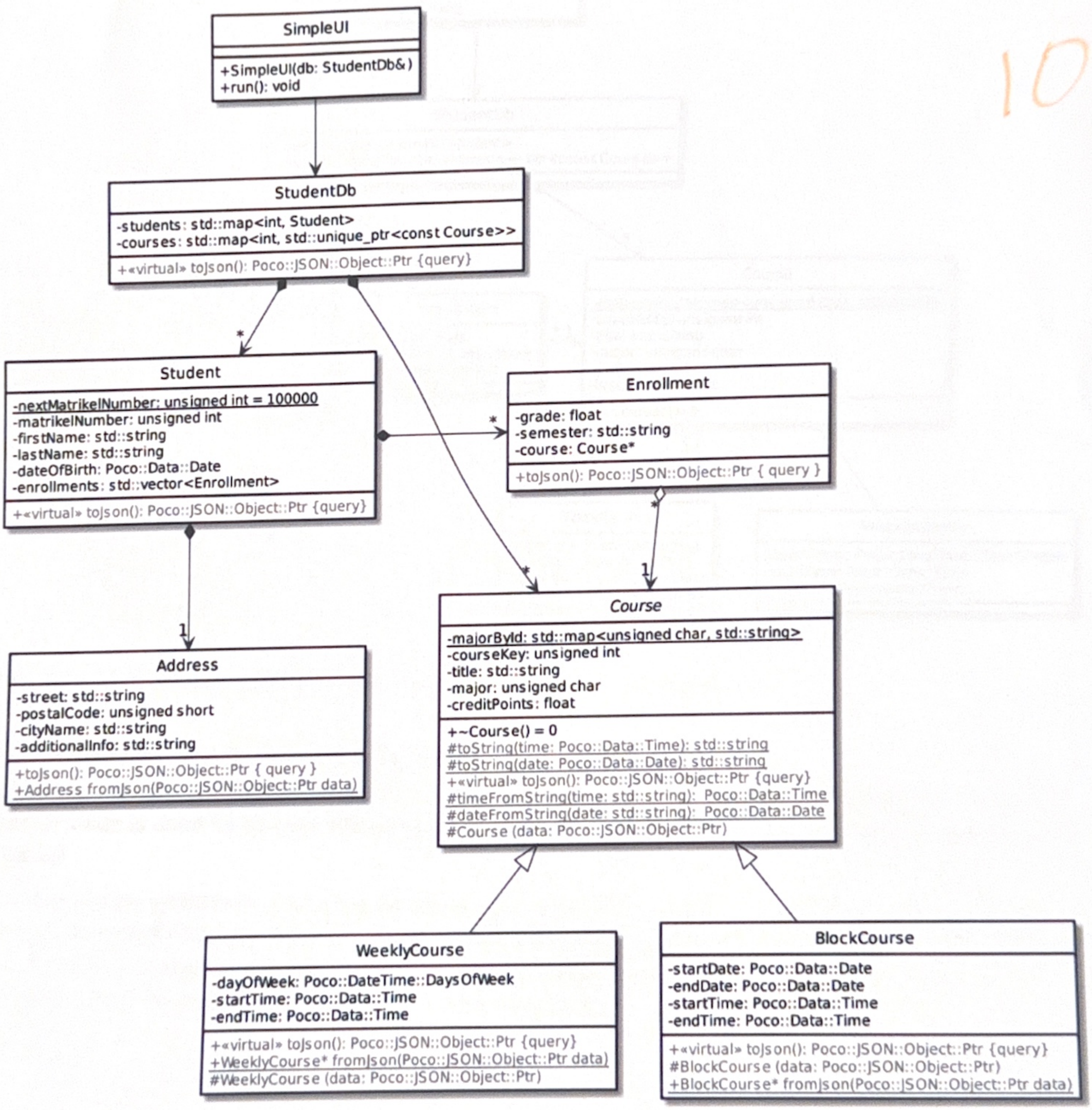
```
class Course {

    /**
     * ...
     *
     * @param semester the semester in the format "WiSe2021", "SoSe2022",
     * "WiSe2022" etc. Case is not checked, i.e.  "wise2021" or even "wISe2022"
     * are acceptable values
     * @throws invalid_argument if the argument violates the proper format
```

2 JSON persistence for the database

In order to store the data from our student data base in CSV-format, we had to "violate" that format quite a bit (by using different numbers of columns for the lines in the file). CSV is not well suited to represent structured data from different items. However, as you have seen when retrieving person data from the server, JSON is well up to that task.

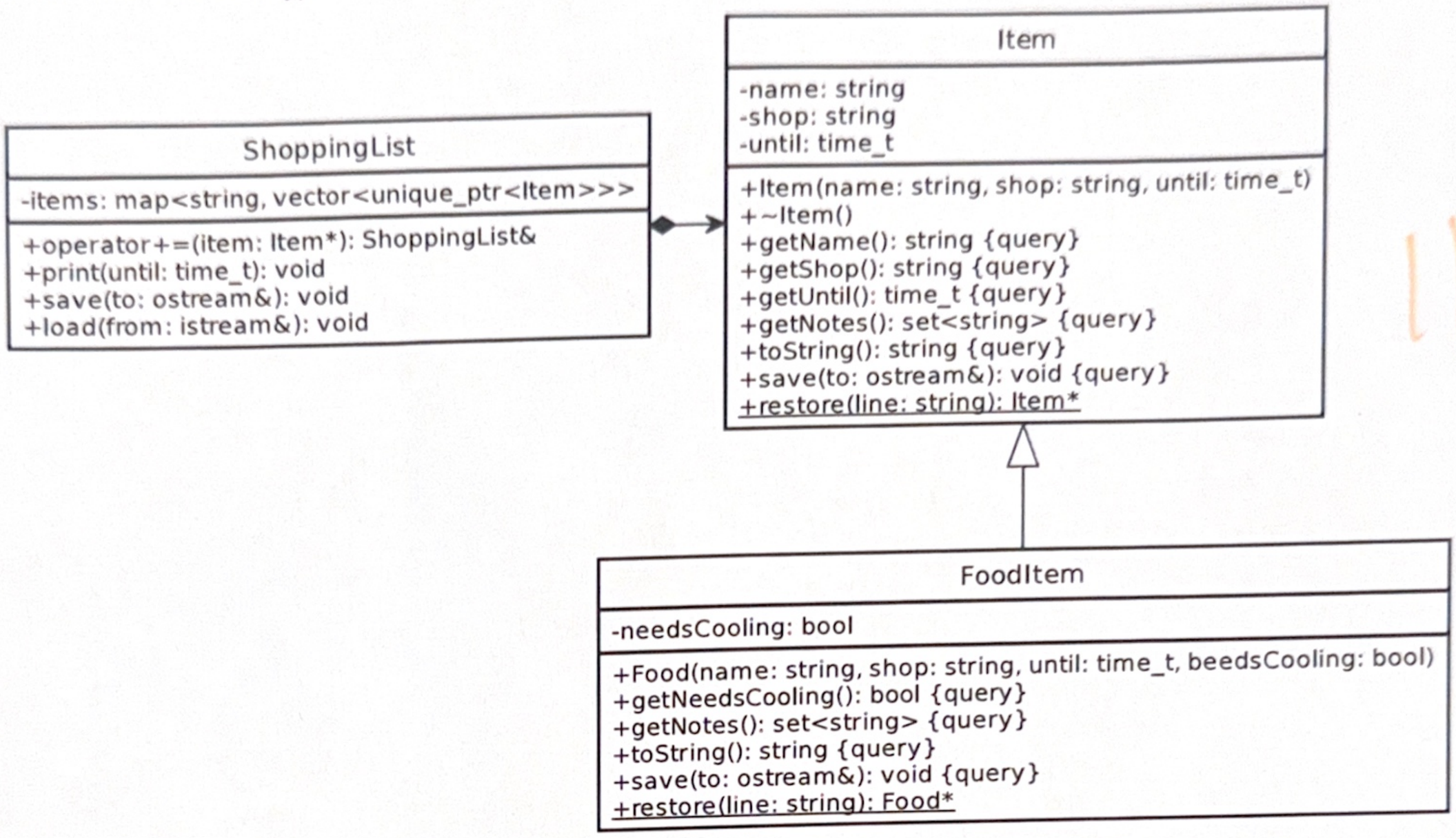
Therefore we now want to implement persistence for the database using the JSON format.



Use (again) your submitted solution for lab 3 as start project for this task.

h_da	Fachbereich Elektrotechnik und Informationstechnik	Advanced Programming Techniques (APT) Module Exam SoSe2020 (PO2019) Prof. Dr. Lipp

Topic of this exam is the modelling of a shopping list. Here's a complete class diagram (`std` prefix omitted for readability):



Reminder: pure virtual (abstract) methods are shown in italics, class methods (static) are underlined.
Note: the diagram has been created "manually" and may have errors. When in doubt, the definition in the code given below takes precedence.

Given is an incomplete Implementation of the shopping list. There are eight methods (marked in red) that you have to implement.

1 Class `Item`

1.1 Header file

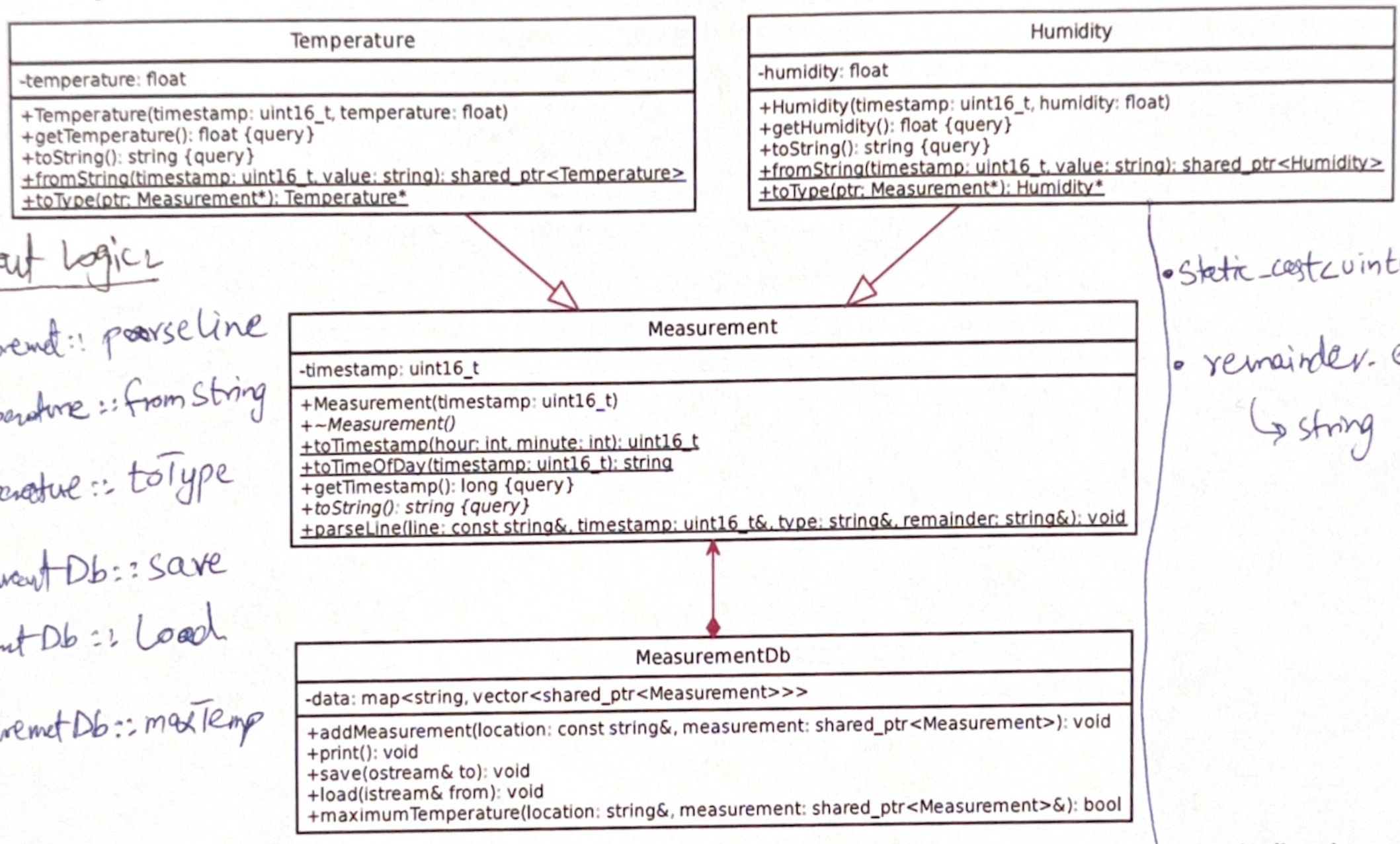
```
#ifndef ITEM_H_
#define ITEM_H_

#include <ctime>
#include <string>
#include <set>
#include <ostream>

/**
 * A shopping list item. Items have a name, they must be bought at
 * a particular shop (bread at the bakery, meat from a butcher etc.),
 * and they are required until a particular date/time. (In reality, some
 * items can be bought at several places --meat from the butcher or from
 * the super market-- but this isn't modelled here.)
 */
class Item {
```

h_da	Fachbereich Elektrotechnik und Informationstechnik	Advanced Programming Techniques (APT) Module Exam WiSe2019 (PO2019) Prof. Dr. Lipp

Topic of this exam is the handling of measurement values from some sensor such as a temperature or humidity sensor. Here's a complete class diagram (`std` prefix omitted for readability) (a larger copy of the diagram can be found on the last page):



Important Logic:

1.) Measurement :: parseline
2.) Temperature :: fromString
3.) Temperature :: toType
4.) MeasurementDb :: save
5.) MeasurementDb :: Load
6.) MeasurementDb :: maxTemp

- static_cast<uint64_t>(value)
- remainder.erase(0, 1)
  ↳ string

12

Reminder: pure virtual (abstract) methods are shown in italics, class methods (static) are underlined.

The recommended way to solve the exercises is to only declare and implement the methods as required (as opposed to immediately declaring all methods in the class definition). This is less error prone. Declarations without definitions yield no points.

Exercise 1 (43 points)

a) Define class `Measurement` with its attribute `timestamp`. The attribute holds the number of minutes since midnight (00:00). Declare and implement the following methods: ◦ `Measurement`: Initialises the attribute with the given value. ◦ `~Measurement`: Does nothing (providing it avoids compiler warning). ◦ `toTimestamp` (class method): Converts the given hour and minute values to the number of minutes since midnight. Throws a `std::invalid_argument` exception if one of the parameter values is out of its valid range. ◦ `toTimeOfDay` (class method): Converts the given timestamp (minutes since midnight) to a string with the format "HH:MM". The hours and minutes are represented with leading zeroes (e.g. "08:05"). (Reminder: Numbers can be converted to `std::string` with `std::to_string`.) ◦ `getTimestamp`: Returns the timestamp value. ◦ `toString`: Must be implemented by the derived classes.	12

Casts

```
Temperature* (Measurement* ptr)
{
    return dynamic_cast<Temperature*>(ptr);
}
```

} To go from parent class pointer to derived class pointer.